

LocalBuka Technical Interview Guide

Purpose and 90-second summary

LocalBuka is a command-line food discovery prototype for Nigerian and continental dishes. A user enters a city and a plain-English request such as “I want something spicy and cheap near me.” The application parses the supported preferences, embeds the request with Gemini, searches a Pinecone vector index for semantically similar dishes, applies structured filters, rechecks dietary requirements locally, and prints the best matching dishes.

I deliberately used a hybrid approach. Gemini embeddings solve the semantic search problem: people can describe a dish in language that does not exactly match the menu. Pinecone makes those vector searches practical as the menu catalogue grows. I did not allow the embedding similarity score to control safety-sensitive decisions by itself. City, cuisine, and maximum price are applied as Pinecone metadata filters, while dietary tags are checked once more in application code after retrieval.

The current catalogue is a hand-curated, Nigeria-first sample of 24 dishes. It includes cities, cuisine, price band, dietary tags, spice, popularity, and a description. The goal is to demonstrate a sound technical design rather than to claim that these are live restaurant records.

Live screen-share plan

Before the interview, confirm that `.env` exists at the `AI_Engineering` root and contains `GEMINI_API_KEY` and `PINECONE_API_KEY`. Do not show this file or either key while sharing your screen.

1. Activate the virtual environment:

```
source /Users/macbook1/Documents/INIE/DATACAMP/venv/bin/activate
```

2. From the `AI_Engineering` directory, run the tests first:

```
cd localbuka_case_study
python -m unittest discover -s tests -v
cd ..
```

Explain that the test suite uses fakes for Gemini and Pinecone. It proves local logic without charging an API or depending on a network connection.

3. Ingest the menu data only if it has changed:

```
python -m localbuka_case_study.ingest
```

This creates or reuses the Pinecone index and upserts the 24 Gemini embeddings. Do not run it repeatedly during the interview unless you are demonstrating ingestion, because it makes 24 embedding API requests.

4. Demonstrate the three scenarios:

```
python -m localbuka_case_study.run_examples
```

5. Demonstrate the interactive assistant:

```
python -m localbuka_case_study.localbuka.cli
```

Enter Lagos, then enter I want something spicy and cheap near me. Also demonstrate one safety behaviour: ask What are the opening hours? and explain that the assistant declines because this sample catalogue does not contain operational data.

Architecture and request flow

```
data.py catalogue
-> ingest.py
-> dish_to_text()
-> GeminiEmbedder.embed_texts()
-> PineconeRestaurantIndex.upsert_dishes()
```

```
CLI user message
```

```

-> FoodAssistant.reply()
-> FoodAssistant.parse_preferences()
-> UserPreferences
-> Recommender.recommend()
-> GeminiEmbedder.embed_one_text()
-> PineconeRestaurantIndex.search() with city/cuisine/price filter
-> local dietary recheck and explanation construction
-> formatted CLI response

```

The ingestion and search paths use the same Gemini embedding model and the same 768-dimension vector configuration. That is essential: a Pinecone index only accepts vectors that match its configured dimension.

Data model: models.py

PRICE_ORDER translates budget, mid, and premium into 1, 2, and 3. Numeric ordering makes a maximum-price check straightforward. VALID_DIETARY_TAGS and VALID_SPICE_LEVELS define the small supported vocabulary and are used for validation.

Dish is a dataclass that represents one catalogue item. Its id is the stable key stored in Pinecone; the other fields describe the restaurant, location, food, filters, popularity, and display description.

UserPreferences is another dataclass. Its list fields use field(default_factory=list) so each user request receives its own empty list. It carries parsed cuisines, maximum price, dietary restrictions, spice preference, cities, and past order IDs.

Recommendation combines a retrieved Dish, its final displayed score, and a list of human-readable reasons. Separating the models from service code keeps the interface clear and makes tests easier to write.

Catalogue: data.py

The dish(...) helper is a short constructor wrapper that keeps the catalogue readable. It receives the ten fields needed for a Dish and returns a Dish object. CATALOGUE is the list of 24 sample dishes. I used primarily Nigerian foods and Nigerian cities because the product context is LocalBuka, not a generic global restaurant app.

The catalogue stores structured fields instead of relying on the embedding to infer every fact. An embedding may identify similarity, but a city, maximum price, or dietary restriction must be stored and checked as explicit product data.

Gemini embeddings: embeddings.py

GeminiEmbedder owns all Gemini API calls. MODEL_NAME is gemini-embedding-2 and DIMENSION is 768. I chose 768 because it provides a compact vector representation that matches the Pinecone index and reduces storage/compute compared with a larger embedding output; in production I would compare retrieval quality at multiple supported dimensions before finalising this choice.

__init__() finds the repository-root .env file with Path, loads it with load_dotenv, reads GEMINI_API_KEY, fails clearly if absent, and creates the Gemini client. Keys are never hardcoded or printed.

embed_texts(texts) loops over the supplied texts. For each text, it calls Gemini's embedding endpoint with output_dimensionality=768 and appends response.embeddings[0].values to the local vectors list. The one-at-a-time loop is intentional: an earlier batch attempt returned one aggregated embedding rather than one embedding per dish. The loop guarantees a one-dish/one-vector mapping, which I verified by checking that the index held 24 vectors after ingestion.

embed_one_text(text) is a convenience method for query time. It passes a one-item list to embed_texts and returns the first vector. It prevents query code from accidentally passing a list of vectors where Pinecone expects one vector.

Pinecone vector store: pine_cone.py

PineconeRestaurantIndex wraps index creation, upsert, and search. Its constants are INDEX_NAME = localbuka-gemini-768, NAMESPACE = localbuka-dishes, and DIMENSION = 768. Keeping these values in one place prevents a mismatch between ingestion and search.

`__init__()` loads `PINECONE_API_KEY`, creates the Pinecone client, calls `_create_index_if_needed()`, and stores an index connection in `self.index`.

`_create_index_if_needed()` first calls `has_index`. If the index is missing, it creates a serverless AWS index in `us-east-1` with cosine similarity and the required 768 dimensions. The readiness loop waits before an upsert is attempted. If it already exists, the method returns; this makes setup idempotent.

`upsert_dishes(dishes, vectors)` joins every Dish with its Gemini vector using `zip`. For each pair, it creates Pinecone metadata: city, cuisine, price rank, dietary tags, spice, popularity, and description. `price_rank` is numeric so Pinecone can perform a less-than-or-equal-to price filter. It then upserts all records into the namespace.

`search(query_vector, preferences, top_k)` creates a Pinecone metadata filter from known preferences. Cities and cuisines use `$in`; price uses `$lte` against `price_rank`. It calls `index.query` with the query vector and returns only IDs and similarity scores. The local application owns display data and performs the second dietary check, rather than trusting external response metadata alone.

`_price_rank(price_band)` is one simple mapping function used while upserting and querying, so the numeric price definition is consistent.

Ingestion: `ingest.py`

`dish_to_text(dish)` creates a single searchable representation of a dish: name, restaurant, city, cuisine, price, dietary tags, spice, and description. This intentionally gives the embedding model useful semantic context while keeping hard filter fields separately structured in metadata.

`main()` builds a `GeminiEmbedder` and `PineconeRestaurantIndex`, converts every catalogue item into text, requests embeddings, and calls `upsert_dishes`. The final print confirms the count submitted. In production, I would trigger this from a menu-change event, batch jobs, retries, and an embedding-version field rather than manually running it.

Recommendation service: `recommender.py`

`Recommender` receives three dependencies—catalogue, embedder, and restaurant index—in its constructor. This is dependency injection: production supplies Gemini/Pinecone objects, while tests supply fakes. The constructor also builds `dishes_by_id`, a dictionary that turns Pinecone match IDs into catalogue records quickly.

`recommend(query_text, preferences, limit=5)` is the main retrieval workflow. It validates input, enriches the query with known past dishes, embeds that text, searches Pinecone for `limit * 5` candidates, then works through the matches. Requesting extra candidates is useful because a candidate can be removed by a later hard dietary check. For each eligible dish, it produces the displayed score and explanations; it stops at limit safe recommendations.

The displayed score is $\text{Pinecone similarity} \times 100 + \text{popularity} \times 0.05$. Semantic similarity dominates; popularity acts only as a small tiebreaker. This score is a prototype convenience, not an offline-trained relevance model.

`_validate(preferences, limit)` rejects impossible requests before an API call: non-positive limits, unknown price bands, unsupported dietary labels, and unsupported spice values.

`_add_order_history(query_text, dish_ids)` turns valid past order IDs into dish names and appends them to the semantic query. This is a modest personalisation signal without training a user model. Unknown IDs are ignored safely.

`_meets_dietary_requirements(dish, restrictions)` checks every requested tag against the local dish record. Any missing tag returns `False`. This is deliberately a post-retrieval hard filter.

`_build_reasons(dish, preferences, similarity_score)` creates transparent reasons: semantic score, city, budget, diet, and exact spice matches. These are used for debugging and could be surfaced in a richer user interface.

Conversational layer: `assistant.py`

`FoodAssistant` is intentionally not a generative chatbot. Its class dictionaries map selected terms to the system vocabulary: for example, `cheap` maps to `budget` and `spicy` maps to `hot`. This limits the assistant to verified catalogue-search functionality.

`__init__` (recommender) stores the recommender and builds a lower-case city lookup from the catalogue. The canonical city values are retained for metadata filtering.

`reply`(message, location=None) is the assistant entry point. It normalises user text; gives a useful example for an empty request; refuses unsupported operational questions; parses preferences; asks for a city when “near me” lacks one; calls `Recommender.recommend`; and formats up to three results. It deliberately avoids claiming hours, address, reservation, delivery, nutrition, or restaurant facts absent from the sample data.

`parse_preferences`(message, location=None) gathers city, cuisine, dietary, price, and spice values. If the user says “near me” and entered a valid city at startup, that city becomes the city filter. It returns a `UserPreferences` object.

`_contains` uses basic substring matching rather than regex because the supported vocabulary is intentionally small. `_matched_values` loops through one vocabulary mapping. `_best_price` returns the lowest stated price band; `_best_spice` gives priority to “not spicy”; `_remove_duplicates` prevents repeated parsed values. These helpers keep `reply` readable and testable.

CLI and examples

`cli.py` is the executable chat entry point. `main()` builds the embedder, Pinecone index wrapper, recommender, and assistant in dependency order. It asks once for a location, loops over user messages, handles quit/exit, and passes every message to `FoodAssistant.reply`.

`run_examples.py` creates three distinct preference profiles: a budget vegan in Lagos, a halal customer in Abuja, and a vegetarian continental-food customer in Lagos. It runs real Gemini/Pinecone searches, so it requires configured keys and an ingested index. This is useful for an interview demo because it demonstrates different filters and outputs.

Tests: `tests/test_localbuka.py`

The tests use Python's built-in `unittest` framework. `FakeEmbedder` records the query text and returns a fixed short vector. `FakeRestaurantIndex` returns predictable IDs and scores. They allow tests to verify the call flow without API cost or network failure.

The tests cover: minimum catalogue size, dietary rechecking after vector retrieval, query forwarding to the embedder, past-order query enrichment, parsing a known “near me” location, asking for location when it is unknown, and refusing unsupported operational data. A useful future extension would add integration tests in a separate, explicitly opt-in suite against a temporary Pinecone namespace.

Key trade-offs and how I would improve it

The main trade-off is sophistication versus operational cost. For a fixed 24-item catalogue, a deterministic rule engine would be cheaper and easier. I chose semantic retrieval to demonstrate how the product could understand varied natural-language food requests and scale to many restaurant descriptions.

With more time, I would:

- Replace sample records with merchant-validated menu data, including availability, delivery area, ingredient/allergen information, price, and freshness timestamp.
- Create labelled relevance data and compare semantic retrieval with keyword search using `Recall@K`, `NDCG@K`, no-result rate, and hard-constraint violation rate.
- Add monitoring for API errors, index vector count, index freshness, ingestion latency, retrieval latency, cost per successful search, and API quotas.
- Embed only changed dishes, batch upserts, cache short-lived repeated searches, and make ingestion asynchronous.
- Add a learning-to-rank stage after candidate retrieval using consented, privacy-preserving signals such as clicks, orders, location, and time of day.
- Add moderation, rate limiting, audit logs without sensitive user text, data-retention controls, and a user feedback/correction path.

Likely interview questions and concise answers

Why Gemini plus Pinecone? Gemini produces semantic text representations; Pinecone provides scalable vector retrieval and metadata filtering. The choice makes natural-language menu search stronger than exact keyword matching.

Why use metadata filters if you have embeddings? Embeddings are good for similarity, not guaranteed compliance. City, price, cuisine, and dietary needs are explicit product constraints and must not be left to probabilistic similarity.

Why is diet checked twice? It is defence in depth. Pinecone stores filter metadata, but local validation makes the final recommendation resilient to stale or malformed index data.

What can go wrong? Stale menu/dietary data, irrelevant semantic matches, API or index outages, bias toward popular restaurants, privacy leaks, and unbounded cost. The mitigations are verified data, offline evaluation, fallbacks, exposure monitoring, data minimisation, quotas, and caching.

What did you debug? I originally saw that a batch embedding call returned a single aggregated vector, which meant only one vector was being stored. I changed ingestion to embed each dish independently, re-ingested the catalogue, checked that the index contained 24 vectors, and verified live search returned results.